Two computations of the height inequality and the multiradial algorithm: reading IUT from the viewpoint of computer science

Toshihisa Urahigashi
ORCID 0009-0004-0460-6242
123026.urahigashi.toshihisa@gmail.com

Version 1.0.0, 5 November 2026
doi:10.5281/zenodo.23968299

Abstract

This note contains almost no new mathematical results. It is an essay that proposes a viewpoint and an idea: to read inter-universal Teichmüller theory (IUT) from the side of computer science. It presents the tools for doing so — two programs that read the same input, and a mockup of the multiradial algorithm in which some parts are left as oracles — and a little analysis made with them. It proves no conclusion. The view that IUT computes the same quantity in two ways is Mochizuki's own and is also recorded in the interim report of Project LANA, and the explicit computation of the IUT side is in Dupuy–Hilado and in Zhou. What this note adds is to place the two computations side by side as programs that read the same input, the mockup with oracles, and a computer-science reading together with a position (§1.3).

The argument has five points. (1) In each of the 36 readings listed by the author and by lana-agents/iut, Corollary 3.12 of IUT III and Theorem 1.10 of IUT IV together give the same height inequality $(k-1)|\log \underline{\underline{q}}| \leq C$ [RF]. (2) This height inequality is written in classical quantities only; for a given curve, both sides can be computed without IUT. (3) What IUT supplies is a different method of computing the two sides; Mochizuki himself describes the development of a mechanism for computing the effect on heights as the fundamental theme of IUT [Alien]. (4) The key is universality. Corollary 3.12 claims to hold for every initial Θ-datum. Since the value for a single datum can be computed classically, what IUT has to add is the "for every". (5) What is supposed to supply this universality is the multiradial algorithm of Theorem 3.11. This note regards it as the essence of IUT, and takes the position that one has not understood IUT essentially unless this algorithm has been elucidated and implemented.

The tools are two. One is a pair of C programs: the classical computation, and the computation of IUT in the reading of IUT IV. The other is a mockup of the multiradial algorithm in C++. In the mockup, the parts that can be implemented at present are implemented: the local invariants from the abstract group $G_{K_v}^{\mathrm{ab}}$, the log-shells, Kummer classes at a finite level, cyclotomic rigidity from the commutator in a theta group, the labels and the tuples of the tensor packets, and the hull of the possible images. The rest is left as named oracles: the anabelian reconstructions, the Kummer classes of the theta values, the global data, the containment of (Ind3), and Step (xi). The values of the oracles are reverse-computed from the ring structure of a theater or from the conclusion, and the output says so. On seven explicit initial Θ-data the mockup agrees with the C programs on 204 items. The implemented part is separated by types so that it cannot read the ring structure of a theater, and its output does not depend on the coordinates that a theater hides. In the family $q \to q^N$, the implemented part runs the same steps for every N, while the reverse-computed Step (xi) answers "no" from $N = 3$ (from $N = 4$ for one datum) on the six completely factored data.

lana-agents/iut has shown that Step (xi) does not follow from the formalized content of Theorem 3.11: over that content Step (xi) is equivalent to the Scholze–Stix inequality, and the missing input is of ABC strength [lana]. This is the result of taking the content of Theorem 3.11, as currently written, as a specification; it does not show that IUT cannot be repaired, as lana-agents/iut itself states. In the view of this note, the key lies in the implementation of the multiradial algorithm. The future task is to implement the oracles as actual algorithms, or as approximations to them. **No claim is made about the truth of IUT, of Corollary 3.12, or of the abc conjecture.**

Contents

1 Introduction **3**
- 1.1 What this note is . . . 3
- 1.2 Where this note stands . . . 3
- 1.3 Prior work . . . 3
- 1.4 The shape of the argument . . . 5
- 1.5 Labels . . . 5

2 In every reading, IUT passes through the height inequality **5**

3 The height inequality is classical **6**

4 What IUT supplies is a different method of computation **7**

5 The key is universality **8**

6 The essence is the multiradial algorithm **8**
- 6.1 Mochizuki's description . . . 8
- 6.2 The position of this note . . . 9
- 6.3 The results of lana-agents/iut and their reach . . . 10

7 Method: comparing the two computations as programs **11**

8 The two programs **12**
- 8.1 The common input . . . 12
- 8.2 The classical computation . . . 13
- 8.3 The computation of IUT . . . 13
- 8.4 Output and cross-check . . . 15

9 A mockup of the multiradial algorithm with oracles **15**
- 9.1 Design . . . 15
- 9.2 What is implemented . . . 17
- 9.3 What is left as oracles . . . 18
- 9.4 Checks and output . . . 19

10 Observations **19**

11 Limitations **22**

12 Future work **23**

13 Reproducing **24**

A Output of the mockup for D211 **24**

1 Introduction

1.1 What this note is

As mathematical work, this note has little content. It contains hardly any theorem or new estimate. What it offers instead is a viewpoint and tools for trying it out. The viewpoint is to compare the way in which IUT computes the height inequality with the classical way, *as programs*. If this is done, the tools of computer science — program equivalence, information flow, relational verification, mockups with oracles — may let one approach the content of IUT.

As the position of this note, two things are stated: "the essence of IUT is the multiradial algorithm" and "one has not understood IUT essentially unless this algorithm has been elucidated and implemented". Neither is a proof; both are statements of a viewpoint (§6.2).

1.2 Where this note stands

This note is also a summary of a series of notes on the computation of the height inequality in IUT. The aim of every one of them has been to grasp the exact algorithm by which IUT computes the height inequality.

Key	Title (version)	Content
[Cat]	Explicit initial Θ-data and the two readings of Step (v) of IUT IV, Theorem 1.10: a catalogue (1.0.0)	seven explicit initial Θ-data and two readings of Step (v)
[Stepv]	A local non-containment in Step (v) of IUT IV, Theorem 1.10 (3.0.0)	a local non-containment in Step (v), conditional on the readings (M), (R)
[SL]	Segment or loop? (1.1.0)	an overview of the dispute over Corollary 3.12 in homotopy type theory
[II]	Reading the IUT dispute through indexed identifications (2.0.0)	an educational model in undergraduate mathematics; readings of (Ind1) and a conservation law (Theorem 14.3)
[WT]	A programmer's walkthrough of inter-universal Teichmüller theory (1.1.0)	a program that runs the parts of IUT that can be written as procedures on explicit data
[Gap]	The gap between the two walls is positive (1.0.0)	a closed form for the (Ind2)-terms of lana-agents/iut; $\Delta(D) > 0$
[RF]	No reading avoids the height inequality (1.0.0)	the same height inequality in each of 36 readings; the conservation of the load

Table 1: the earlier notes (all on Zenodo, CC BY 4.0).

1.3 Prior work

Much of the viewpoint of this note has precedents. We state first how much is new (the search for prior work was done on 5 November 2026).

S *The view that there are two computations is Mochizuki's own.* Remark 3.11.1 of IUT III speaks of the construction of Theorem 3.11 as an algorithm and calls the theta values its hidden internal structures. It also distinguishes Corollary 3.12, which concerns only the qualitative logical consequences of the algorithm, from the explicit computation of IUT IV, §1, which uses essentially the way in which theta values occur:

> "That is to say, theta functions/theta values may be regarded as HIS of the construction algorithm of Theorem 3.11 — somewhat like the internal structure of the CPU or operating system of a computer!" ... "On the other hand, we observe that, unlike Corollary 3.12 below, which only concerns qualitative logical aspects/consequences of the construction algorithm of Theorem 3.11, the explicit computation to be performed in [IUTchIV], §1, of

the log-volumes that occur in the statement of Corollary 3.12 makes essential use of the way in which theta values occur in the construction algorithm of Theorem 3.11."

IUT III, Remark 3.11.1, (iv) [IUT3]

X The interim report of Project LANA records the same view as Mochizuki's:

"According to Mochizuki, however, IUT provides two distinct ways of computing the log-volume of the q-pilot object: the usual computation and the computation via the multiradial algorithm."

LANA interim report, §10.3 [LANA]

X *The explicit computation of the IUT side* is in Dupuy–Hilado [DH]. They take the average over the tuples of the tensor packets and over the labels as an "iterated expectation", and obtain the average $\ell(\ell+1)/12$ of j^2 over the labels, the average $\frac{\ell+5}{4}$ of the number of factors, and the Szpiro constant $6+\varepsilon_\ell$ with $\varepsilon_\ell = (24\ell+72)/(\ell^2+\ell-12) = O(1/\ell)$. Observations (ii)–(iv) of §10 are a re-check of this computation in the form of programs. Zhou gives an algorithm computing an estimate of the same form and has turned it into Python and C++ code [Zhou].

"we can view the expectation of X as an 'iterated expectation' "

Dupuy–Hilado, §7.1 [DH]

"Then there exists an algorithm [whose construction is presented in the proof] to compute a real number $\mathrm{Vol}(\mathfrak{R}) \geq 0$ that depends only on $\mathfrak{R}$"

Zhou, Corollary 1.13 [Zhou]

X *Separating a finite layer from a structural layer* is in Maeki [Maeki], who isolates the layer of coefficients of the explicit estimates of IUT IV that can be checked finitely, and delegates the structural layer of IUT (Hodge theaters, Frobenioids, mutually alien copies, log-volume inequalities) to an assumption IUT-Struct. The mockup of §9, which names the parts that cannot be computed and leaves them as oracles, divides in the same way.

"the IUT-Explicit-ADIC isolates the Σ_1-verifiable finite content of the explicit IUT estimates, while delegating the truth of the full structural IUT layer to the IUT-Struct assumption."

Maeki, the abstract of the Zenodo record [Maeki]

S *The meaning of "algorithm".* A "group-theoretic algorithm" in the sense of IUT is a construction phrased in a language that depends only on the topological group structure of a fundamental group, not a procedure that runs on a computer [AbsTopIII]. X Joshi objects that it is not a finitistic procedure and that it is highly ineffective [Joshi]. What this note calls an "implementation" (§6.2) means closing this gap.

"We use the term 'group-theoretic algorithm' to mean that the algorithm in question is phrased in language that only depends on the topological group structure of the fundamental group under consideration."

Mochizuki, Topics in absolute anabelian geometry III, Remark 1.9.8 [AbsTopIII]

"But 'algorithm' as used in (Mochizuki, 2021) is not a finitistic procedure as commonly understood by mathematicians." … "Furthermore, these procedures are highly ineffective, notably they do not produce the claimed outputs, and do not certifiably produce outputs which can be distinguished from each other (this is necessary in the proof)."

Joshi, comment (5) [Joshi]

H *What this note adds* is limited to three things. (a) Placing the classical computation and the computation of IUT side by side, as two programs that read the same input; Dupuy–Hilado and Zhou derive and compute the IUT side, but do not place the two on the same input. (b) A mockup in which a part of the multiradial algorithm is left as oracles whose values are reverse-computed, with a separation of information flow by types, a check of non-interference, and a scan of the family $q \to q^N$. (c) A computer-science reading (non-interference, relations over all inputs) and the position that "one has not understood IUT essentially unless the multiradial algorithm has been elucidated and implemented". Mochizuki himself uses a word close to "non-interference" in a mathematical sense (§6.1); we do not claim that the reading of (c) is the same as his.

1.4 The shape of the argument

1. **In every reading, IUT passes through the height inequality** (§2). A summary of [RF] and of lana-agents/iut [lana].
2. **The height inequality is classical** (§3). Both sides are written in classical quantities and can be computed for a given curve without IUT. Whether it holds for all curves is open (a form of the Szpiro conjecture).
3. **What IUT supplies is a different method of computing the two sides** (§4). We quote Mochizuki and state our reading of him.
4. **The key is universality** (§5). Since the value for a single datum can be computed classically, what the proof needs is the part "for every initial Θ-datum".
5. **The essence is the multiradial algorithm, and the key is its implementation** (§6). Mochizuki's description, the results of lana-agents/iut and their reach, and the position of this note.
6. **The two computations compared as programs, and the multiradial algorithm as a mockup with oracles** (§§7–9).
7. **Observations, limitations and future work** (§§10–12).

Points 1 and 2 are facts that can be checked against earlier results and computations; 3 and 5 are statements of the sources and our reading of them; 4 is a fact that can be checked by computation together with our reading; 6 is a proposal of method and a mockup. None of them decides whether IUT is right or wrong.

1.5 Labels

S a statement of the sources (IUT and Mochizuki's documents); X a claim of others (lana-agents/iut, Joshi, Scholze–Stix, ...); P a fact or computation shown in this note or in the earlier notes; H our reading, interpretation, proposal or position (not a proof).

2 In every reading, IUT passes through the height inequality

Corollary 3.12 compares two log-volumes. One is the log-volume $-|\log \underline{q}|$ of the q-pilot, which is not subject to the indeterminacies. The other is the log-volume $-|\log \underline{\underline{\Theta}}|$ of the holomorphic hull of the union of the possible images of the Θ-pilot, which is subject to the indeterminacies (Ind1), (Ind2), (Ind3) [IUT3]. The proof of Theorem 1.10 of IUT IV bounds $-|\log \underline{\underline{\Theta}}|$ from above [IUT4]. This gives the chain

$$-|\log \underline{q}| \;\le\; -|\log \underline{\underline{\Theta}}| \;\le\; U(D).$$

Putting the two ends together, with $a = |\log \underline{\underline{q}}|$, $k = \ell(\ell+1)/12$ (the average of j^2 over the labels $j = 1, \dots, \ell^*$) and $C = U(D) + k\,a$, gives the height inequality

$$(k-1)\,a \;\le\; C.$$

P The value of the middle term $-|\log \underline{\underline{\Theta}}|$ depends on the "reading". The readings have five axes: the range of the label permutations of (Ind1), container or iteration for (Ind3), the group of (Ind2), whether the hull is taken, and the archimedean container. [RF] showed the following in each of the 36 readings obtained by combining the axes found so far.

- Corollary 3.12 and the upper bound of IV together give the same height inequality $(k-1)a \le C$, because the two ends a, k, C are computed by one procedure that takes no reading, and only the middle term depends on the reading (a conservation law).
- A reading decides only how much of the wall $(k-1)a$ Corollary 3.12 carries and how much IV carries. No reading makes both loads nonpositive.
- In the family obtained by replacing q with q^N, the statement that holds for every N carries no positive load, and the other carries at least the whole wall. Choosing the reading differently does not make the proof cheaper.

X lana-agents/iut [lana] collects various formulations of the wall and shows that they are equivalent in its models (`Iut/WallEquiv/`). For one reading it also shows that Step (xi) does not follow from the formalized content of Theorem 3.11 alone (`NonDerivable.lean`; §6.3). It proves, with no other hypothesis left, that the variant of Corollary 3.12 for all initial Θ-data gives the abc conjecture (`Iut/MainTheorem.lean`).

S The route of the sources is Theorem 3.11 → Corollary 3.12 → Theorem 1.10 of IV → ABC. Theorem 1.10 of IV draws its conclusion, in its own statement, by applying the inequality of Corollary 3.12. The sources contain no route that avoids Corollary 3.12.

> "and hence, by applying the inequality '$C_\Theta \ge -1$' of [IUTchIII], Corollary 3.12, conclude that"
>
> IUT IV, Theorem 1.10 [IUT4]

H Within the readings listed by the author and by lana-agents/iut, therefore, whoever derives ABC through IUT has to let someone carry the height inequality (its load), in every reading. This is what "IUT passes through the height inequality" means.

X There is a position that denies this premise. Joshi holds that the Tate parameter changes together with (Ind1), (Ind2), (Ind3) [Joshi]. On that view the left end also depends on the reading, and the conservation law does not apply. This note follows the wording of Corollary 3.12, under which $-|\log \underline{\underline{q}}|$ is not regarded as subject to the indeterminacies [IUT3].

3 The height inequality is classical

S The inequality that Theorem 1.10 of IV finally gives has the form [IUT4]

$$\tfrac{1}{6}\cdot\log(\mathfrak{q}) \;\le\; \left(1+\tfrac{20\cdot d_{\mathrm{mod}}}{l}\right)\cdot\left(\log(\mathfrak{d}^{F_{\mathrm{tpd}}})+\log(\mathfrak{f}^{F_{\mathrm{tpd}}})\right)+20\cdot(e^*_{\mathrm{mod}}\cdot l+\eta_{\mathrm{prm}}),$$

where $\log \mathfrak{q}$ is the normalized degree of the divisor of the q-parameters, $\log \mathfrak{d}$ that of the different (the normalized logarithm of the discriminant), $\log \mathfrak{f}$ that of the conductor, and d_{mod}, e^*_{mod}, η_{prm} are constants.

S Mochizuki himself summarizes the result of this computation as a bound on the height of the form expected from the Szpiro conjecture:

> "This computation may be summarized as asserting that one obtains a bound on the height of the elliptic curve under consideration whose leading term is the 'log-different $\log(\mathfrak{d}^K)$' [cf. the final portion of §3.7, (iv)] that one expects from [a suitable version of] the Szpiro Conjecture."
>
> Mochizuki, The mathematics of mutually alien copies, §3.8, (11^{gau}) [Alien]

P In either form, only classical quantities occur in the inequality: the degree of the q-parameters, the discriminant and the conductor, the different of the field K, logarithms of primes, and constants. The term C of the intermediate form $(k-1)a \leq C$ of [RF] is also a closed expression in local classical quantities, as observation (ii) of §10 shows. To compute both sides for a given curve, one finds $\mathrm{ord}_v(q)$ at each place from the valuations of the j-invariant of the curve and adds up degrees. The program of §8.2 does just this.

H That both sides can be computed is different from the inequality holding for all curves. The latter is a form of the Szpiro conjecture and is open by methods that do not use IUT. When this note says "the height inequality is classical", it means that *its statement is written in classical quantities only and its value for each datum can be computed classically*, not that it has been proved classically.

4 What IUT supplies is a different method of computation

S Mochizuki explains in terms of computation what IUT is a theory of. He casts the problem of heights in the form "$N \cdot h = h$" [or the inequality "$N \cdot h \leq h + C$"] and then writes:

> "One then verifies the desired approximate relation q^N '$\approx$' q by computing $\{q_{\text{LHS}}^{j^2}\}_j$ — not in terms of q_{LHS} [which is immediate from the definitions!], but rather — in terms of [the scheme theory surrounding] q_{RHS} [which is a highly nontrivial matter!]."
>
> Mochizuki, The mathematics of mutually alien copies, Introduction [Alien]

> "Of course, such an assignment is not compatible with the ring structure of an NF, hence does not exist in the framework of conventional scheme theory." ... "in some sense the fundamental theme of inter-universal Teichmüller theory consists of the development of a mechanism for computing the effect — e.g., on heights of elliptic curves [cf. the discussion of §2.3!] — of such non-scheme-theoretic 'Gaussian Frobenius morphisms' on NF's."
>
> Mochizuki, The mathematics of mutually alien copies, §2.4 [Alien]; "such an assignment" is $\underline{\underline{q}} \mapsto \{\underline{\underline{q}}^{j^2}\}_{j=1,\ldots,l^*}$

> "Then the N-th power map of (i), (ii) corresponds, after passing to heights, to multiplying real numbers by N; the multiradial algorithm corresponds to saying that the height is unaffected (up to a mild error term!) by multiplication by N, hence that the height is bounded!"
>
> Mochizuki et al., On the formalization of IUT: a preliminary progress report, §3 [Form]

H Put side by side, these statements let us read the picture of IUT as follows.

- If the log-volume of the Θ-pilot $\{\underline{\underline{q}}^{j^2}\}_j$ is computed on the original side (q_{LHS}), it is $-k\,a$ directly from the definitions. This is the "classical" computation; it only multiplies the height by k.
- IUT computes the same quantity on the far side of the Θ-link (the scheme theory surrounding q_{RHS}), namely in the tensor packets of log-shells, allowing indeterminacies. The result is $-|\log \underline{\underline{\Theta}}|$, whose value is $-k\,a$ plus what Mochizuki calls an "error factor" (the container, the unit part, the hull and so on: the remainder B of [RF]).

- Corollary 3.12 asserts that the value so computed is at least the value $-a$ of the q-pilot. Since $k > 1$, this does not hold by the classical computation alone. If it holds, it holds because of the error (B) brought in by the other method of computation. This is the height inequality itself, in the form $N \cdot h \leq h + C$.

H In one sentence, our reading is this. The core of the proof of IUT is to give a "different way of grasping" an open inequality (the height inequality) by decomposing both of its sides into a method of computation different from the existing one, and thereby to attempt a proof that could not be given in the existing framework. In Mochizuki's own words it is the development of a "mechanism for computing". If this reading is right, the content of IUT lies in the *difference* between the two methods of computation.

5 The key is universality

S The statement of Corollary 3.12 begins with the sentence

> "Suppose that we are in the situation of Theorem 3.11."
>
> IUT III, Corollary 3.12 [IUT3]

The situation of Theorem 3.11 is built from one initial Θ-datum. Corollary 3.12 claims to hold whatever that initial Θ-datum is.

P For one given initial Θ-datum, on the other hand, both sides of the height inequality can be computed classically (§§3, 8). For the six data of this note that are completely factored the inequality holds with a large margin (Table 2); for D2357 the counted places alone leave it short, and the verdict depends on the places above its unfactored cofactor [RF, §5.5]. What IUT has to add is therefore not the value for one datum, but the part that it holds for *every* initial Θ-datum.

X lana-agents/iut shows in Lean that this "for every" is exactly where the strength of ABC lies. The variant of Corollary 3.12 for all initial Θ-data gives the abc conjecture (`classicalABC_of_variant`), and so does Step (xi) for all genuine data (`classicalABC_of_forall_xi`).

> "**The missing input is ABC-strength**: `classicalABC_of_forall_xi` (Step (xi) for all genuine data gives `ClassicalABC`, via `classicalABC_of_thm311`)."
>
> lana-agents/iut, `Iut/WallEquiv/README.md` [lana]

P The family $q \to q^N$ of [RF] shows the same thing. The statement that holds for every N (the upper bound of IV) carries no positive load, and the whole wall falls on Corollary 3.12. The same happens in the mockup of §9 on this family (§9.4).

H In terms of computer science, Corollary 3.12 is a property over all of infinitely many inputs (a relation between two outputs). Trying finitely many inputs (testing) never establishes it, however many are tried; an argument about the program (verification) is needed. IUT seeks to obtain it from the *uniformity* of the construction of Theorem 3.11, that is, from the fact that the same procedure applies to every initial Θ-datum. The step that does this is Step (xi) of the proof of Corollary 3.12. The question "does IUT prove ABC?" thus becomes the question "where does the 'for every' of Corollary 3.12 come from?". The answer of IUT is: from the multiradial algorithm.

6 The essence is the multiradial algorithm

6.1 Mochizuki's description

S The sentence of the progress report quoted in §4 says that it is the "multiradial algorithm" that states that the height is unaffected by multiplication by N [Form]. The same report also states what the algorithm takes as input:

> "Main point: The multiplicative monoid and abstract group structures (but not the ring structures!) are common (cf. 'logical AND $\wedge$!') to $\dagger$, $\ddagger$ and can be used as the input data for an algorithm to construct the multiradial representation for the Θ-pilot, i.e., a common container for the distinct ring strs. (i.e., 'arith. hol. strs.')"
>
> Mochizuki et al., On the formalization of IUT: a preliminary progress report [Form]

H Restated in terms of computer science, the multiradial algorithm is a procedure that reads only the inputs common to the two sides (multiplicative monoids and abstract groups) and produces an output (a common container) that does not depend on the ring structures of the two sides. If the ring structures are regarded as secret inputs, this is the requirement of *non-interference* in information-flow security (the output does not depend on the secret inputs). The mockup of §9 expresses this separation in the types of the program.

S Mochizuki himself uses the phrase "mutually non-interference" in a mathematical sense, for the Kummer theories on the two sides of the Θ-link:

> "the mutually non-interference or simultaneous executatibility of the Kummer theories surrounding the q-pilot and Θ-pilot" … "when the Kummer theory surrounding the q-pilot is held fixed, and one allows the Kummer theory surrounding the Θ-pilot to be subject to various indeterminacies."
>
> Mochizuki, On the essential logical structure of IUT, §3.5, (NonInf) [EssLgc]

X The interim report of Project LANA lists as open the question whether the value-group part affects the output of the algorithm:

> "As suggested here, the input to the 'raw' Theorem 3.11 algorithm appears to consist solely of the étale-unit BPS, and the value-group part does not seem to affect the algorithm's result." … "Reconciling these points remains a topic of discussion among LANA members."
>
> LANA interim report, §6.2, Remark 6.2.2 [LANA]

H The "non-interference" of this note is the property of information-flow security (the output does not depend on secret inputs). Mochizuki's (NonInf) states that two Kummer theories are compatible; we do not claim that the two are the same. The open question of LANA, on the other hand, has exactly the form of a question of non-interference: whether the output depends on an input, the value group. The mockup of §9 puts questions of this kind into a form that can be posed in types and checks.

6.2 The position of this note

H This note regards the multiradial algorithm as the essence of IUT, for four reasons.

1. It is this algorithm that actually performs the "different method of computation" of §4: its content is to compute the Θ-pilot again on a side that does not share the ring structure.
2. The universality of §5 is supposed to come from the uniformity of this algorithm. Corollary 3.12 begins from "the situation of Theorem 3.11", and Step (xi) makes the comparison.
3. The other steps are classical or elementary. The estimate of IV is a local computation of log-shells and differents (§8.3), and the route from Corollary 3.12 to ABC has been formalized in Lean by lana-agents/iut (§2).
4. The C program of IUT in §8 only computes the estimate of IV of the output of this algorithm; it is not the algorithm. Both sides come out without it. The part that could not be written in the program is exactly the part of the algorithm.

S Mochizuki regards the step from Theorem 3.11 to Corollary 3.12 ("Stage 1") as the essential nontrivial content of the theory. He also writes that one can understand the delicate inter-relations of the parts of the structures only by working with specific algorithms:

> "We chose to concentrate on this aspect of the theory first since this aspect of the theory — i.e., 'Stage 1: [IUTchIII] Theorem 3.11 $\Longrightarrow$ Corollary 3.12' (cf. the discussion at the beginning of §2) — has received the most public attention. Indeed, this aspect corresponds to the essential nontrivial content of the theory, i.e., that the height of the elliptic curve under consideration is equal to N times the height of the elliptic curve (where N is a large positive number), up to a small discrepancy (arising from (Ind1,2,3) + hull), thus implying a bound on the height."
>
> Mochizuki et al., On the formalization of IUT: a preliminary progress report [Form]

> "That is to say, although different algorithms, or software, may yield the same output data, it is only by working with specific algorithms that one may understand the delicate inter-relations that exist between various components of the structures that occur as one performs various operations [i.e., the mutations of a mutation-history]." "In the case of the theory developed in the present series of papers, one central example of this phenomenon is the cyclotomic rigidity isomorphisms that underlie the theory of $\Theta^{\times\mu}_{\mathrm{LGP}}$-link compatibility discussed in [IUTchIII], Theorem 3.11, (iii), (c), (d)"
>
> IUT IV, Remark 3.6.2, (i) [IUT4]

H Mochizuki takes Stage 1 as the essential content, and this note takes the multiradial algorithm, which carries it, as the essence. The difference is one of emphasis; we agree that what should make Stage 1 hold is the uniformity of the algorithm (§5). Mochizuki also names the cyclotomic rigidity isomorphisms as the central example of the relations that only specific algorithms reveal. The first thing implemented in the mockup of §9 is a finite model of cyclotomic rigidity.

H This note therefore takes the following position. **One has not understood IUT essentially unless the multiradial algorithm has been elucidated and implemented.** Here an "implementation" means giving the procedure that computes the output from the data common to the two sides alone, either in a form that runs on finite data or as a construction that can be verified formally. Either way, whether the output satisfies Step (xi) can then be asked of the implementation.

6.3 The results of lana-agents/iut and their reach

X lana-agents/iut has shown that Step (xi) does not follow from the formalized content of Theorem 3.11:

> "**Theorem 3.11 of IUT III, as formalised here, does not imply Step (xi) of the proof of Corollary 3.12.**" … "a proof of Step (xi) for all genuine `D` must use facts about `D` beyond them, and such a proof would give the ABC conjecture"
>
> lana-agents/iut, the module docstring of `Iut/WallEquiv/NonDerivable.lean` [lana]

X Over its package, Step (xi) is equivalent to the Scholze–Stix inequality $(s-1)\,|\log q| \le$ log-vol(hull($VC(I)$)) (`xi_iff`; s is the k of this note). This is itself a height inequality, with the volume of the container on the right. Moreover the package is realized for all values of a and V (`family`). Hence no inequality bounding a follows from the formalized content of Theorem 3.11 (that the package exhausts what is proved at the level of Theorem 3.11 is, as lana-agents/iut itself says, audited and not a theorem). lana-agents/iut also writes:

> "The repository proves no other relation coupling `V` with `a`." … "**Not shown**: that Step (xi) fails for some genuine `D`, or that ABC fails; the package is a model"
>
> lana-agents/iut, the module docstring of `Iut/WallEquiv/NonDerivable.lean` [lana]

X The plan for formalizing the whole of Theorem 3.11 in the same repository (a draft of 14 October 2026) says that what exists is a theorem about a model and not the algorithm, and that a genuine Theorem 3.11 would not by itself decide Step (xi):

> "We have IUT III Thm 3.11 as a theorem *about a model*. We do not have the algorithm." … "It does not decide Cor 3.12, Step (xi)." … "A genuine 3.11 would turn that from a statement about our models into a statement about IUT's actual objects. It would not prove or refute SS."
>
> lana-agents/iut, `Iut/Multiradial/ROADMAP.md`, §0 [lana]

H From the above, this note reads the non-derivability of lana-agents/iut as follows.

- What lana-agents/iut has shown is *the result of taking the content of Theorem 3.11, as currently written, as a specification.* Neither Step (xi) nor any height inequality follows from that specification. But this is not a proof that IUT has suffered fatal damage that cannot be repaired. lana-agents/iut itself writes that it has not shown that Step (xi) fails for some genuine datum.
- This note holds that the key lies in the implementation of the multiradial algorithm. Step (xi) can be asked of the actual objects of IUT, rather than of a model, only once an implementation exists (the "statement about IUT's actual objects" of the plan of lana-agents/iut).
- The strength of what would have to be found there is, however, fixed: a uniform property of ABC strength that is not written in the specification. In the words of lana-agents/iut, a relation coupling the size V of the possible images with $a = |\log \underline{\underline{q}}|$, or facts about genuine data beyond the package. One may not expect an implementation to fill a small gap; an implementation only makes visible whether the actual construction has such a property. In the mockup of §9, the place where it would enter is the oracle `[step-xi]`.
- There is an opposite position. Scholze–Stix hold that if the identifications of the copies of the real numbers are taken consistently, the inequality of Corollary 3.12 becomes empty [SS, §2.2]. On that view, an implementation will not produce the required property.

7 Method: comparing the two computations as programs

H To study the difference between the two methods of computation, we propose to place them side by side as two programs that read the same input, instead of comparing them in the language of difficult mathematics. There are three reasons.

1. **The flow of data becomes explicit.** A program writes, line by line, what it reads, what it builds, and what it compares where. A reader who does not share the conventions of the mathematics can follow which input affects which output.
2. **The two are compared on the same ground.** If the two programs are given the same input (the local data of the same curve), the only difference is the path of the computation.
3. **The tools of computer science can be used**, for example:
   - program equivalence and loop summarization (symbolic evaluation): are the two computations the same function of the input, and if not, where do they differ?
   - analysis of data flow and information flow: along which paths does the size of q reach the output? Multiradiality can be written as non-interference with respect to a secret input, the ring structure (§6.1).
   - relational verification: a framework that treats claims of the form "for all inputs, the output of program A $\leq$ the output of program B" as something to prove rather than to test. Corollary 3.12 has this form (§5, §10 (v)).

- mockups with oracles: the parts that cannot be implemented are named as oracles, and where their values come from is recorded, so that the code distinguishes what is implemented from what is not (§9).
- adversarial families of inputs: instead of checking on inputs that happen to have a large margin, one tries families of inputs that make the wall large (the family $q \to q^N$ of [RF]).

S Mochizuki himself has a caveat on what this method cannot reach. When the Gaussian integral is computed in polar coordinates, it is a hopeless task to identify which portion of the original one-dimensional integral "corresponds" to the radial integral. In the same way he writes:

> "it is essentially a hopeless task to trace, in some sort of explicit or readily computable fashion, the way in which the value group portions (b^{Θ}), (c^{Θ}) in the domain of the Θ-link [cf. (9^{gau})] appear within the multiradial representation via tensor packets of log-shells [cf. (10^{gau})] expressed in objects arising from the codomain of the Θ-link"
>
> Mochizuki, The mathematics of mutually alien copies, §3.8 [Alien]

H This is not a statement that the tools of computation are insufficient, but a structural statement: the corresponding pieces do not exist in any meaningful form. This note therefore does not look for a line-by-line correspondence between "this line" of the classical computation and "this line" of the computation of IUT. What we compare is the shape of the whole computation: what is read as input, what intermediate structures are built, where q enters, and which comparison remains, uncomputed, as a claim. What Mochizuki says cannot be traced is the content of the comparison of Corollary 3.12. In the mockup of §9 it lies inside the oracle `[step-xi]`.

8 The two programs

8.1 The common input

P The input is the local data of one initial Θ-datum: the prime ℓ, the field $F = \mathbb{Q}(\sqrt{D})$, and, for each place of F above a prime dividing $N(a)N(1-a)$ (for the curve $y^2 = x(x-1)(x-a)$), the prime p, the residue degree f, the ramification index e_F of $F_v/\mathbb{Q}_p$ and $\mathrm{ord}_v(q)$ (0 for good reduction). For example, the file for D211 begins

```
# local data of the initial Theta-datum D211: E: y^2 = x(x-1)(x-a) over F = Q(sqrt(5))
# written by export_local_data.py from elementary_height.py; the same as the places of iut_compute.py
name D211
ell 7
D 5
unfactored 0
# place  prime  f  e_F  ord_q   (one line per place of F above a prime dividing N(a)N(1-a))
place 2 2 2 1 0
place 3 3 2 1 2
place 5 5 1 2 2
place 59+ 59 1 1 0
place 59- 59 1 1 2
place 211+ 211 1 1 58
place 211- 211 1 1 0
...
```

The local data were taken from an elementary program that does not use IUT (`elementary_height.py`, which finds $\mathrm{ord}_v(q)$ from the valuations of the j-invariant). We checked that the places it gives agree, prime by prime, as multisets of $(f, e_F, \mathrm{ord}\, q)$, with those built independently by the program of the walkthrough [WT], for all seven data (`export_local_data.py`). The two C programs and the C++ mockup all read this same file.

8.2 The classical computation

P The classical computation uses only the places of bad reduction, their residue degrees and $\mathrm{ord}_v(q)$. No labels, tensor packets or log-shells occur. The constants are copied from the statement of Theorem 1.10; its proof is not used.

```
/* classical.c -- the height inequality of IUT IV, Theorem 1.10, computed directly from the curve.
 *
 * Uses only the places of bad reduction, their residue degrees and the orders of q: no labels, no tensor packets,
 * no log-shells. The constants are copied from the statement of the theorem; nothing of its proof is used.
 *
 *     (1/6) log q  <=  (1 + 20 d/l) (log d + log f) + 20 (e* l + eta)
 *
 * log q, log f, log d: the normalized degrees of the q-parameter divisor, of the conductor (the same support with
 * coefficient 1) and of the discriminant of F;  d = [F:Q] = 2,  e* = 2^12 3^3 5 e  with e = 2,  eta = 2.
 *
 * usage: classical <datum file>
 */
#include "localdata.h"

/* the places counted for the comparison with IUT: V^bad of the catalogue minus the places above 3 and 5 */
static int counted(const Datum *d, const Place *v)
{
    long long p = v->small;
    return v->ordq > 0 && p != 2 && p != 3 && p != 5 && p != d->ell && v->ordq % d->ell != 0;
}

int main(int argc, char **argv)
{
    if (argc != 2) { fprintf(stderr, "usage: classical <datum file>\n"); return 2; }
    Datum d = read_datum(argv[1]);
    const double deg = 2;                                   /* [F:Q] */
    const double estar = 4096.0 * 27 * 5 * 2, eta = 2;
    double log_q = 0, log_f = 0, log_q_all = 0, log_f_all = 0;
    double log_d = log_disc_F(d.D);

    for (int i = 0; i < d.n; i++) {
        const Place *v = &d.v[i];
        if (v->ordq == 0) continue;                         /* good reduction */
        log_q_all += v->f * v->ordq * v->logp / deg;        /* all places of bad reduction */
        log_f_all += v->f * v->logp / deg;
        if (!counted(&d, v)) continue;
        log_q += v->f * v->ordq * v->logp / deg;
        log_f += v->f * v->logp / deg;
    }
    double rhs = (1 + 20 * deg / d.ell) * (log_d + log_f) + 20 * (estar * d.ell + eta);

    printf("datum %s  l = %d%s\n", d.name, d.ell, d.unfactored ? "  (a cofactor is not factored)" : "");
    printf("all places of bad reduction: log q = %.10f  log f = %.10f\n", log_q_all, log_f_all);
    printf("counted places:              log q = %.10f  log f = %.10f  log d = %.10f\n", log_q, log_f, log_d);
    printf("Theorem 1.10: (1/6) log q = %.10f %s %.12e = right-hand side\n", log_q / 6, log_q / 6 <= rhs ? "<=" : ">",
    ↪  rhs);
    printf("Szpiro ratio log q / (log d + log f) = %.10f\n", log_q / (log_d + log_f));
    return 0;
}
```

To compare with the IUT side, the places counted are those of $\mathbb{V}_{\mathrm{mod}}^{\mathrm{bad}}$ of the catalogue [Cat] minus those above 3 and 5 (as in [Gap, §5], and [WT]; Definition 3.1, (b), of IUT I allows a subset). The values over all places of bad reduction are printed too.

8.3 The computation of IUT

P From the same input, the computation of IUT first builds the structure that IUT attaches to each place:

- the weight $w_v = [F_v : \mathbb{Q}_p]/[F : \mathbb{Q}]$ (summing to 1 over the places above p);
- membership in $\mathbb{V}_{\mathrm{mod}}^{\mathrm{bad}}$;
- the ramification index $e_K = e_F \cdot 15\ell/\gcd(15\ell, \mathrm{ord}\, q)$ of $K = F(E[\ell])$, with $F = F_{\mathrm{mod}}(\sqrt{-1}, E[15])$ (at bad places; above p the formula of Appendix A of the catalogue, assumed at the other bad places);

- the order of $\underline{\underline{q}} = q^{1/2\ell}$, the outer radius ρ of the log-shell $(1/2p)\log(\mathcal{O}_v^\times)$, and the order $(e-1)/e$ of the different.

Then, for each prime carrying theta values, it enumerates the labels $j = 1, \dots, \ell^*$ and the tuples $t = (t_0, \dots, t_j)$ of places, one for each factor of the tensor packet (weight $w(t_0) \cdots w(t_j)$), and for each tuple computes

- the hull on the Θ-side (the reading of IV: the theta value sits on the factor j; container, units, hull, small archimedean part): $\sum_i \rho(t_i) - j^2 \operatorname{ord}(\underline{\underline{q}}_{t_j})$;
- the local estimate of Proposition 1.4, (iii), of IV with $\lambda = j^2 \operatorname{ord}(\underline{\underline{q}}_{t_j})$: $(-\lambda + \sum_i \mathfrak{d}(t_i) + 1)\log p + \sum_{e(t_i)>p-2}(3 + \log e(t_i))$.

These are averaged with the weights of the tuples and over the labels (the procession normalization). The places above 2, 3, 5, ℓ are not computed: on the side of the hull their contribution, which is nonnegative, is set to 0, and on the side of the upper bound they are bounded by Step (ii) (the different) and (R4) (the terms of I^*). Finally the archimedean part $\frac{\ell+5}{4}\log\pi$ is added, and the program goes on to Step (viii) and the form of Theorem 1.10. The central enumeration:

```
    /* 3. The primes that carry theta values (a place in V^bad), one at a time. For each label j and each tuple
     *    T = (t_0, ..., t_j) of places above p (one for each factor of the tensor packet, weight w(t_0)...w(t_j)):
     *      Theta-side: the hull of the container, with the theta value on the factor j:
     *                  sum_i rho(t_i) - j^2 rq(t_j)                                              [units of log p]
     *      IV:         Proposition 1.4 (iii) with lambda = j^2 rq(t_j):
     *                  (-lambda + sum_i dif(t_i) + 1) log p + sum over i with e(t_i) > p - 2 of (3 + log e(t_i))
     *    Average over the tuples (weighted) and over the labels (procession normalization). */
    double theta_sum = 0, iv_sum = 0, closed_sum = 0, logd_known = 0;
    int done[MAXPLACES] = {0}, enumerated = 1;
    for (int i0 = 0; i0 < d.n; i0++) {
        if (done[i0] || !L[i0].bad) continue;
        int idx[MAXP], m = 0;
        for (int i = 0; i < d.n; i++)
            if (same_prime(&d.v[i], &d.v[i0])) { idx[m++] = i; done[i] = 1; }
        const double logp = d.v[i0].logp;
        const long long p = d.v[i0].small;
        double theta_p = 0, iv_p = 0, wsum = 0;
        for (int j = 1; j <= ls; j++) {
            double th = 0, iv = 0;
            if (pow(m, j + 1) <= (1 << 22)) {                   /* all m^(j+1) tuples */
                int t[128] = {0};
                for (;;) {
                    double wt = 1, rho = 0, dif = 0, star = 0;
                    for (int i = 0; i <= j; i++) {
                        const Loc *u = &L[idx[t[i]]];
                        wt *= u->w; rho += u->rho; dif += u->dif;
                        if (p > 0 && u->eK > p - 2) star += 3 + log((double)u->eK);
                    }
                    double lambda = (double)j * j * L[idx[t[j]]].rq;   /* the theta value sits on the factor j */
                    th += wt * (rho - lambda);
                    iv += wt * ((-lambda + dif + 1) * logp + star);
                    int i = 0;                                  /* next tuple */
                    while (i <= j && ++t[i] == m) t[i++] = 0;
                    if (i > j) break;
                }
            } else {                                            /* the same average, by linearity */
                double rbar = 0, dbar = 0, sbar = 0, qbar = 0;
                enumerated = 0;
                for (int s = 0; s < m; s++) {
                    const Loc *u = &L[idx[s]];
                    rbar += u->w * u->rho; dbar += u->w * u->dif; qbar += u->w * u->rq;
                    if (p > 0 && u->eK > p - 2) sbar += u->w * (3 + log((double)u->eK));
                }
                th = (j + 1) * rbar - (double)j * j * qbar;
                iv = (-(double)j * j * qbar + (j + 1) * dbar + 1) * logp + (j + 1) * sbar;
            }
            theta_p += th * logp / ls;
            iv_p += iv / ls;
        }
        /* the same prime in closed form: (l+5)/4 (log d_p + sum_{I*} w (3 + log e)) - (l+1)/24 log q_p + log p */
        double logd_p = 0, star_p = 0, logq_p = 0;
        for (int s = 0; s < m; s++) {
            const Loc *u = &L[idx[s]];
            wsum += u->w;
```

```
            logd_p += u->w * u->dif * logp;
            logq_p += u->w * 2 * l * u->rq * logp;
            if (p > 0 && u->eK > p - 2) star_p += u->w * (3 + log((double)u->eK));
        }
        if (fabs(wsum - 1) > 1e-12) { fprintf(stderr, "weights above %s do not add up to 1\n", d.v[i0].p); return 1; }
        closed_sum += (l + 5) / 4.0 * (logd_p + star_p) - (l + 1) / 24.0 * logq_p + logp;
        logd_known += logd_p;
        theta_sum += theta_p;
        iv_sum += iv_p;
    }
```

8.4 Output and cross-check

	classical		IUT				
datum	$\log \mathfrak{q}$	$\log \mathfrak{f}$	$-\lvert\log \underline{q}\rvert$	$-\lvert\log \underline{\underline{\Theta}}\rvert$ (IV)	$U(D)$	wall $(k-1)a$	C
D211	305.21	77.68	−21.801	135.26	1145.40	79.94	1247.14
D109	267.60	68.12	−19.114	116.65	1098.32	70.09	1187.52
D2	240.09	76.92	−17.149	158.00	1155.16	62.88	1235.19
D3	1128.24	270.90	−80.589	449.24	1871.53	295.49	2247.61
D18	614.95	160.86	−43.925	285.92	1478.09	161.06	1683.07
D15	516.07	137.29	−36.862	243.31	1384.66	135.16	1556.68
D2357*	3717.84	30.23	−11.840	−22675.23	−8246.15	24463.93	16229.62

Table 2: output of the two programs (counted places). D2357* is conditional, since the places above its unfactored cofactor are not counted; for its counted places the wall exceeds C.

P For every datum, $2\ell \cdot |\log \underline{q}|$ on the IUT side agrees with $\log \mathfrak{q}$ on the classical side, and the two sides of the inequality of Theorem 1.10 that the two programs finally print agree (the right-hand side is about 1.548×10^8 for $\ell = 7$ and about 3.47×10^9 for D2357). We compared the output of the two C programs with the Python programs of the bundle of [RF] (`elementary_height.py`, `iut_height.py`, `iut_readings36.py`) on the seven data: 119 items agree within a relative error of 10^{-9} (`check_c.py`). The items are, on the classical side, $\log \mathfrak{q}$ and $\log \mathfrak{f}$ (over all places and over the counted ones), $\log \mathfrak{d}$ and the right-hand side of the theorem; on the IUT side $-|\log \underline{q}|$, $-|\log \underline{\underline{\Theta}}|$, $U(D)$ with its parts, Step (viii) and the right-hand side of the theorem; and the two ends against each other.

9 A mockup of the multiradial algorithm with oracles

P The program of IUT in §8 only computes the estimate of IV of the output of the multiradial algorithm. The C++ mockup of this section makes a place for the algorithm. The parts that can be implemented at present are implemented, and the rest is left as oracles. The values of the oracles are reverse-computed from the ring structure of a theater or from the conclusion. That they are reverse-computed is written in the comments of the code, in the marks of the output (such as `[ORACLE step-xi]`) and the word `REVERSE-COMPUTED`, and in a ledger at the end of the output. The oracles are named by their content rather than numbered (Table 5). **The output of the mockup proves nothing.**

9.1 Design

P The mockup builds two Hodge theaters †, ‡ from the same initial Θ-datum. The ring side of a theater (the ramification of K_v, the valuations of q, and the coordinates that the theater chose on its cyclotome Δ_Θ) is private. The implemented part of the multiradial algorithm receives only the data common to the two sides and the étale-like data of †. The header `common.hpp` that defines them does not know the class of the theater. The file of the algorithm,

`multiradial.cpp`, does not include the header of the theater, even indirectly, so code that reads the ring of a theater there does not compile.

file	role	reads the ring of a theater?
`common.hpp`	everything that crosses from a theater to the algorithm: types of abstract groups, Kummer classes, boxes of theta groups, global data	no
`theater.*`	the ring side of a Hodge theater; all private	itself
`oracle.*`	the six oracles; they reverse-compute their values and enter them in a ledger	yes (friends of the theater)
`multiradial.*`	the implemented part (A)–(G) of the multiradial algorithm	cannot (headers checked transitively)
`iv.*`	the estimate $U(D)$ of IV, Step (viii), the form of Theorem 1.10	cannot
`link.*`	the Θ-link: checks that the data common to the two theaters agree place by place, and links the Θ-pilot of † to the q-pilot of ‡; not a map of rings	only through the oracles
`classical.hpp`	the classical computation (as in §8.2)	reads the local data of the curve directly
`main.cpp`	report, check of non-interference, scan of the family q^N	—

Table 3: the files of the mockup (16 files, 1,170 lines in all).

The flow of the computation is: initial Θ-datum → theaters †, ‡ (each choosing its own coordinates) → Θ-link → oracles `[gab]`, `[global]`, `[kummer]`, `[theta-group]` (reverse-computing the étale-like data of †) → implemented part (A)–(G) → oracle `[ind3]` → the q-pilot of ‡ → oracle `[step-xi]` (reverse-computing Step (xi)) → Corollary 3.12 → the estimate of IV → the height inequality and Theorem 1.10. What reaches the algorithm (a part of `common.hpp`):

```
// ORACLE [theta-group]. The theta group at level N as a black box: the algorithm may multiply, invert and ask for the
↪  two
// generators, nothing else. It models the theta quotient of the tempered fundamental group of the Tate curve
// ([EtTh] Section 2; lana-agents/iut, Iut/EtaleTheta/Heisenberg.lean), whose commutator pairing identifies the two
// cyclotomes. Here the box is built from the theater's own choice of coordinates on Delta_Theta.
class ThetaGroupBox {
public:
    virtual ~ThetaGroupBox() = default;
    virtual ThetaGroupElement mul(const ThetaGroupElement& x, const ThetaGroupElement& y) const = 0;
    virtual ThetaGroupElement inv(const ThetaGroupElement& x) const = 0;
    virtual ThetaGroupElement generator_t() const = 0;   // the generator of the Z-part
    virtual ThetaGroupElement generator_a() const = 0;   // a generator of the mu_N-part
};

struct EtalePlace {                    // one place of the dagger theater, with everything the algorithm may read
    std::string name;
    AbelianizationType gab;            // ORACLE [gab]
    bool bad = false;                  // in V^bad: part of the initial Theta-datum
    std::vector<KummerClass> theta;    // ORACLE [kummer]: Kummer classes of the theta values qq^(j^2), j = 1, ...,
    ↪  l*,
                                       //            in the coordinates of Delta_Theta (bad places only)
    std::shared_ptr<const ThetaGroupBox> theta_group;   // ORACLE [theta-group] (bad places only)
};

struct GlobalData {                    // ORACLE [global]: in IUT from the copies of F_mod (the global Frobenioids)
    int ell = 0;                       // the prime l of the initial Theta-datum
    std::vector<std::string> prime_label;   // places with the same label lie above the same prime
    std::vector<double> weight;        // [F_v : Q_p] / [F : Q]
    double log_disc_F = 0;             // log d^F of F = F_mod (= F_tpd for the Legendre form)
    double log_cond_F = 0;             // log f^F over V^bad
};

struct AlgorithmInput {                // the dagger theater's etale-like data and the global data
    GlobalData global;
    std::vector<EtalePlace> places;
};

struct QPilotInput {
    ...
};
```

9.2 What is implemented

step	what it does	basis
(A)	recovers p, d, f, e from the type of the abstract group $G_{K_v}^{\mathrm{ab}} \cong \widehat{\mathbb{Z}} \times \mathbb{Z}_p^d \times$ (torsion): p is the prime with an extra $\mathbb{Z}_p$-part, d its rank, f is read off from the order $p^f - 1$ of the torsion prime to p, and $e = d/f$	local class field theory; the plan of lana-agents/iut also lists the local invariants of G_K as already genuinely constructed
(B)	the outer radius ρ of the log-shell and the order of the different, from (p, e)	the log-shells of IV, §1 (the formulas of §8.3)
(C)	cyclotomic rigidity at level $N = 2^{61} - 1$: recovers the coordinates that the theater chose on Δ_Θ from the commutator $[t, a]$ in the box of the theta group	the remark of the paper on the étale theta function quoted below, and the Heisenberg group of `Iut/EtaleTheta/Heisenberg.lean` of lana-agents/iut
(D)	reads the valuations of the theta values $\underline{\underline{q}}^{j^2}$ from their Kummer classes written in the coordinates of Δ_Θ, and checks the structure $\kappa(\theta_j) = j^2\kappa(\theta_1)$	Kummer theory at a finite level (the units are N-th powers)
(E)	the procession: labels $j = 1, \dots, \ell^*$ and the tuples of places of the tensor packets	as in §8.3
(F)	possible images and their hull; in the reading of IV, (Ind1) = average over the labels, (Ind2) = units, (Ind3) = container	as in §8.3
(G)	the archimedean container $\frac{\ell+5}{4} \log \pi$	as in §8.3

Table 4: the implemented part (`multiradial.cpp`).

S Step (C) is a model, at a finite level, of the following point that the paper on the étale theta function makes about cyclotomic rigidity:

> "One further interesting aspect of cyclotomic rigidity in the theory of the present paper is that it is obtained essentially as the result of a certain computation involving commutators [cf. Remark 2.19.2]. Put another way, one may think of this cyclotomic rigidity as a sort of consequence of the nonabelian structure of [what essentially amounts, from a more classical point of view, to] the 'theta-group'."
>
> Mochizuki, The étale theta function and its Frobenioid-theoretic manifestations, Introduction [EtTh]

P The theta group of the mockup is the product $(c, a, b)(c', a', b') = (c + c' + b\,a',\ a + a',\ b + b')$ of the Heisenberg group of lana-agents/iut, written in the coordinates that the theater chose on Δ_Θ. The algorithm may use only multiplication, inversion and the two generators. The c-coordinate of the commutator $[t, a_1]$ gives the unit γ of the theater's coordinates directly.

```
// (C) cyclotomic rigidity at level N. In the theta group the commutator pairing a |-> [t, a] identifies the
// cyclotome mu_N of the Z-covering with the central cyclotome Delta_Theta ([EtTh] Prop. 2.12; lana-agents/iut,
// Iut/EtaleTheta/Heisenberg.lean, cyclotomicRigidity_aut). The c-coordinate of [t, a_1] is the coordinate of the
// image of the generator a_1: the unit by which the theater's coordinates on Delta_Theta differ from those of mu_N.
u64 rigidity_unit(const ThetaGroupBox& G)
{
    const ThetaGroupElement t = G.generator_t(), a = G.generator_a();
    const ThetaGroupElement c = G.mul(G.mul(t, a), G.mul(G.inv(t), G.inv(a)));
    if (c.a != 0 || c.b != 0 || c.c == 0) throw std::runtime_error("the commutator is not a nonzero central element");
    return c.c;
}

    ...

    // (C)-(D) The theta values qq^(j^2) at the bad places, from their Kummer classes in Delta_Theta.
    for (int i = 0; i < n; i++) {
        const EtalePlace& P = in.places[i];
        if (!P.bad) continue;
        if (!R.local[i].known || !P.theta_group || (int)P.theta.size() != ls)
```

```
            throw std::runtime_error("incomplete etale-like data at " + P.name);
        const u64 g = use_rigidity_ ? rigidity_unit(*P.theta_group) : 1;
        const u64 ginv = invmod(g);
        R.rigidity[i] = g;
        std::vector<u64> val(ls);                    // ord_{K_v}(qq^(j^2)) mod N
        for (int j = 1; j <= ls; j++) val[j - 1] = mulmod(P.theta[j - 1].value, ginv);
        for (int j = 1; j <= ls; j++)
            if (val[j - 1] != mulmod(val[0], (u64)j * j)) R.gaussian_ok = false;
        R.theta_ord[i] = (double)val[0] / R.local[i].e;   // ord_p(qq) = ord_{K_v}(qq) / e; the valuations are << N
    }
```

9.3 What is left as oracles

name	content in IUT	how the mockup fills the value	why it cannot be implemented now
`[gab]`	the abstract group G_v (an infinite profinite group)	reverse-computes the type $(p, [K_v : \mathbb{Q}_p], p^f - 1)$ from the field K_v by local class field theory	there is no way to hand over a profinite group as finite data; the mockup hands over the type itself, so p is written in the input
`[global]`	the copies of F_{mod} (global Frobenioids) and their Kummer theory: weights, labels of primes, $\log \mathfrak{d}^F$, $\log \mathfrak{f}^F$	reverse-computed from the number field F and $\mathbb{V}^{\mathrm{bad}}_{\mathrm{mod}}$	depends on global anabelian reconstruction (Belyi cuspidalization); by the plan of lana-agents/iut it presupposes the relative p-adic Grothendieck conjecture, whose proof uses p-adic Hodge theory
`[kummer]`	the Kummer class of the étale theta function and the Kummer classes of the theta values obtained by its Galois evaluation; the Kummer classes of the q-pilot of ‡	reverse-computed at level N from the valuations of q in the field (the theta values written in the coordinates of Δ_Θ)	tempered fundamental groups and the étale theta function cannot be held as finite objects
`[theta-group]`	mono-theta environments (the theta quotient of the tempered fundamental group)	builds the box of a Heisenberg group at level N from the theater's coordinates on Δ_Θ	the same; but the computation of the rigidity from the box, step (C), is implemented
`[ind3]`	upper semi-compatibility of (Ind3) (IUT III, Proposition 3.5, Theorem 3.11, (ii), (a))	only assumed (the possible images are the containers)	no iteration of log-links and no log-Kummer correspondence
`[step-xi]`	Step (xi) of the proof of Corollary 3.12: the q-pilot of ‡ lies in the hull of the possible images	returns the result of comparing the two log-volumes of the datum: **reverse-computed from the conclusion**	this is the disputed part; it does not follow from the formalized content of Theorem 3.11 (§6.3)

Table 5: the oracles (`oracle.cpp`; every value is reverse-computed).

P The oracle for Step (xi) is shown below. What it compares is the conclusion itself, and the comment and the returned value say so.

```
StepXiCertificate step_xi(const Representation& rep, const QPilot& qp)
{
    StepXiCertificate X;
    X.margin = rep.minus_log_theta - qp.minus_log_q;
    X.contained = X.margin >= 0;
```

```
    X.how = "reverse-computed from the conclusion: the two log-volumes of this datum were compared. A Step (xi) "
            "derived from Theorem 3.11 would give the containment for every initial Theta-datum without comparing "
            "numbers";
    OracleLedger::record("step-xi", "Step (xi): the q-pilot of ddagger lies in the hull of the possible images",
    ↪  X.how);
    return X;
}
```

9.4 Checks and output

- P **Cross-check.** On the seven data the output of the mockup was compared with that of the C programs; 204 items agree within a relative error of 10^{-9} (`check_cpp.py`). Since the C programs agree with the Python programs on 119 items (§8.4), the mockup agrees with the Python programs too. The items are the two ends, $-|\log \underline{\Theta}|$, $U(D)$ with its parts, the wall, C, Step (viii), the two sides of Theorem 1.10, the values of the classical side, and, prime by prime, the enumerated and the closed values of the hull.
- P **Information flow.** Following the headers transitively with `g++ -H`, we checked that `multiradial.cpp` and `iv.cpp` include the headers of the theater, the oracles and the link not even indirectly (`make check`).
- P **Non-interference.** When the coordinates that the theaters choose on Δ_Θ (a unit of $\mathbb{Z}/N$ per place) are changed in eight ways, the output is the same bit for bit (all seven data). Without step (C) the output depends on that choice ($-|\log \underline{\Theta}| \approx -5.1 \times 10^{18}$ for D211, and so on), although the j^2-structure is still seen.
- P **The family $q \to q^N$.** We ran $N \le 40$ prime to ℓ (Table 6). The implemented part runs the same steps for every N. The reverse-computed Step (xi) (the oracle `[step-xi]`) answers "yes" only for small N, for all six data; the height inequality holds up to much larger N. This family consists in general not of initial Θ-data of actual curves (it is an adversarial input).

datum	N for which `[step-xi]` answers "yes"	N for which the height inequality holds
D211	$N \le 2$ ("no" from $N = 3$)	$N \le 15$ (fails from $N = 16$)
D109	$N \le 2$ ("no" from $N = 3$)	$N \le 16$ (fails from $N = 17$)
D2	$N \le 3$ ("no" from $N = 4$)	$N \le 19$ (fails from $N = 20$)
D3	$N \le 2$ ("no" from $N = 3$)	$N \le 6$ (fails from $N = 8$)
D18	$N \le 2$ ("no" from $N = 3$)	$N \le 10$ (fails from $N = 11$)
D15	$N \le 2$ ("no" from $N = 3$)	$N \le 11$ (fails from $N = 12$)

Table 6: the family $q \to q^N$ ($N \le 40$, N prime to ℓ). D2357 was not run; for it the oracle `[step-xi]` answers "no" already at $N = 1$ (Table 2).

The output of the mockup for D211, its check of non-interference and its scan of $q \to q^N$ for $N \le 20$ are in Appendix A.

10 Observations

What follows is what one sees by reading the two programs and the mockup side by side. None of it decides whether IUT is right or wrong.

(i) Input, the structures built, and the two ends. P The two programs start from the same input. The IUT side builds many structures from it: the ramification indices of K, weights, labels, tuples of places, radii of log-shells, differents, and the estimates above $2, 3, 5, \ell$. But all of them are determined by the input, and no new input is read. The numbers at the two ends ($\log \mathfrak{q}$ and the right-hand side of Theorem 1.10) agree (§8.4). The classical side reads

only f and $\operatorname{ord} q$ at the places of bad reduction and the discriminant of F; the places of good reduction and e_F are read only by the IUT side (to build the weights and the ramification of K).

(ii) Folding the right-hand side. P In the reading of IV, every term of the enumeration of tuples is a sum of terms that depend on one factor only. The weighted average therefore folds into a closed expression: for each prime p,

$$U_p = \tfrac{\ell+5}{4}\Big(\log \mathfrak{d}_p^K + \sum_{e_v > p-2} w_v(3 + \log e_v)\Big) - \tfrac{\ell+1}{24}\log \mathfrak{q}_p + \log p,$$
$$M_p = \tfrac{\ell+5}{4}\sum_v w_v \rho_v \log p - \tfrac{\ell+1}{24}\log \mathfrak{q}_p,$$

where $\frac{\ell+5}{4}$ is the average over the labels of the number $j+1$ of factors and $\frac{\ell+1}{24}\log \mathfrak{q}_p = k\, a_p$. The C program prints U_p, and the mockup prints both U_p and M_p, as enumerated and in closed form; the two agree on the six data with $\ell = 7$ (for D2357 the long tuples are counted by linearity). X This folding is done by hand by Dupuy–Hilado (the iterated expectation over tuples and labels, and the average $\frac{\ell+5}{4}$ of the number of factors) [DH].

H That is, as a function of the input, the right-hand side of the computation of IUT is a closed expression in local classical quantities ($\log p$, ramification indices, $\operatorname{ord} q$, weights). The enumeration of labels and tuples makes no new numbers; it rearranges classical quantities. The intermediate form $(k-1)a \le C$ is, in this sense, an inequality in classical quantities only. This holds in the reading of IV. In other readings (L, L0, which enlarge the label permutations), terms taking a minimum enter, and the closed expression becomes an order statistic of the distribution of the orders of q ([II, Proposition 13.2]; [RF]).

(iii) How q enters, and the coefficient of the wall. P On the classical side, q enters as $\log \mathfrak{q}$ directly. On the IUT side, the size of q enters in two places: the q-pilot $-|\log \underline{\underline{q}}| = -\log \mathfrak{q}/(2\ell)$, and the theta values $j^2 \operatorname{ord}(\underline{\underline{q}})$. Averaged over the labels, the latter becomes $-k\,a$, and enters the hull of the Θ-side and the upper bound of IV in the same form. The remaining terms (log-shells, differents, I^*, $2, 3, 5, \ell$, the archimedean part) depend on the orders of q only through where the reduction is bad, whether ℓ divides them, and $\gcd(15\ell, \operatorname{ord} q)$ (the ramification of K), not through their size.

H Hence the coefficient of the wall $(k-1)a$ is decided by the average k of j^2 over the labels alone. The classical side has nothing corresponding to this k. The "Gaussian Frobenius morphism" $\underline{\underline{q}} \mapsto \{\underline{\underline{q}}^{j^2}\}_j$ of Mochizuki (an assignment not compatible with the ring structure) appears in the program as this one coefficient k.

(iv) Where the constant 6 comes from. P The constant $\frac{1}{6}$ on the classical side is copied from the statement of Theorem 1.10. On the IUT side, the ratio of the coefficient $(k-1)/(2\ell)$ of the wall (on $\log \mathfrak{q}$) to the coefficient $\frac{\ell+5}{4}$ of the leading term of the margin (on $\log \mathfrak{d}^K$) gives

$$c_\ell = \frac{\ell+5}{4}\cdot\frac{2\ell}{k-1} = \frac{6\ell(\ell+5)}{(\ell+4)(\ell-3)},$$

in the form $\log \mathfrak{q} \le c_\ell \log \mathfrak{d}^K + \cdots$, with $c_7 = 11.45$, $c_{157} = 6.15$, and 6 in the limit $\ell \to \infty$. X This constant is in Dupuy–Hilado, as $\frac{1}{6+\varepsilon_\ell}$ with $\varepsilon_\ell = (24\ell+72)/(\ell^2+\ell-12)$, where they also note $\varepsilon_\ell = O(1/\ell)$ [DH]; $6+\varepsilon_\ell = c_\ell$. The estimate of Zhou has the same factor $(\ell^2+5\ell)/(\ell^2+\ell-12) = c_\ell/6$ [Zhou]. Observation (iv) is not a new result; it rereads this in the language of programs.

H In terms of programs, the constant 6 is the ratio of the averages of two loops: the average over the labels j of the number $j+1$ of factors (how many times the term of the different is

added), and the average of the weights j^2 of the theta values minus 1 (with what multiple q enters).

(v) The comparison that remains uncomputed. P The two programs only compute the two sides for a given datum; they do not prove the inequality. In the programs, Corollary 3.12, the core of the proof of IUT, is just the one-line comparison $-|\log \underline{\underline{q}}| \le -|\log \underline{\underline{\Theta}}|$. In the reading of IV, the value on the Θ-side is $-k\,a + B$ with $B = M + k\,a = \frac{\ell+5}{4}(\sum_p \bar{\rho}_p \log p + \log \pi) +$ [a nonnegative quantity above $2, 3, 5, \ell$]. So Corollary 3.12 becomes $(k-1)a \le B$, the height inequality itself with the radii of the log-shells as margin ("in the reading of IV, Corollary 3.12 carries the whole wall" of [RF]).

H In terms of computer science, Corollary 3.12 is a relation (a relational property) between the outputs of two programs (the q-side and the Θ-side) *over all inputs*. On the six completely factored data the comparison holds because the wall is small, 5–13% of C, and the margin is large (Table 2; D2357 falls short for its counted places and is conditional). As a test, nothing has been tried. Turning this into a proof is supposed to be the task of the multiradial algorithm of Theorem 3.11 (§§5, 6).

(vi) What the oracle `[step-xi]` hides. P In the mockup, the only place where "for every initial Θ-datum" can enter is the oracle `[step-xi]`. The implemented part (A)–(G) computes the same function of the local invariants on actual data and on the data of $q \to q^N$. The margin B of the hull does not grow with $|\log \underline{\underline{q}}|$ (it moves slightly through $\gcd(15\ell, \operatorname{ord} q)$; for D211, C takes two values, 1247.14 and 1235.66). Hence on the family $q \to q^N$ the oracle `[step-xi]` answers "no" from $N = 3$ (from $N = 4$ for D2).

H If a genuine Step (xi) holds for all genuine data, it must be using something that the implemented part does not have: a property that couples the size of the possible images with $|\log \underline{\underline{q}}|$, or facts that distinguish genuine data from arbitrary local data. This is the same as lana-agents/iut's "no other relation coupling V with a" and "facts about D beyond them" (§6.3), seen in the program. When the oracles are replaced by implementations, this is what remains last.

(vii) Cyclotomic rigidity fixes the scale. P Without step (C), the j^2-structure of the Kummer classes ($\kappa(\theta_j) = j^2\kappa(\theta_1)$) is still seen, but the size of q is not: the values become multiples of the unit chosen by the theater, and the output swings on the order of 10^{18}. With step (C), the output does not depend on the choice of coordinates.

H Within the finite model, cyclotomic rigidity works as what fixes the scale when étale-like classes are turned into numbers. That it is obtained from the structure of a group alone, the commutator in the theta group, agrees with the remark of the paper on the étale theta function (quoted in §9.2). This is, however, a finite model of the algebraic core, not the cyclotomic rigidity of IUT itself (§11).

(viii) Multiradiality as non-interference. H The description in the progress report (a common container built from the inputs common to the two sides alone; §6.1) is, in terms of computer science, a requirement of non-interference. In the mockup this is enforced for the implemented part by the structure of the headers and checked by changing the coordinates of the theaters. The oracles, on the other hand, read the rings of the theaters. Replacing an oracle by an implementation means removing that reading of the ring while keeping the output; the separation by types then becomes a tool for checking mechanically that the replacement is non-interfering.

11 Limitations

- **Not all readings are covered.** The 36 readings are the combinations of the axes found so far by the author and by lana-agents/iut. We do not claim that they exhaust the readings of the sources. Moreover, the C program and the C++ mockup implement only one reading, that of IV (the 36 readings are in the Python programs of [RF]).
- **A computation is not a proof.** That both sides can be computed, and that the inequality holds on the six completely factored data, shows nothing about the correctness of IUT or about the Szpiro conjecture.
- **The mockup is not the construction of IUT.** The theaters, the Θ-link and the theta group are finite models, not the objects of IUT (Frobenioids, tempered fundamental groups, the log-theta-lattice). The cyclotomic rigidity is also only a finite model of the algebraic core, the commutator in a theta group. (Ind1), (Ind2), (Ind3) are the formulas of the reading of IV, not the indeterminacies of Theorem 3.11 themselves.
- **The values of the oracles are reverse-computed.** In particular, the line "Corollary 3.12 (from the oracle step-xi)" of the output of the mockup puts the conclusion in and takes the conclusion out. The mockup only shows where the algorithm has to enter and what is needed there.
- **Assumptions of the finite level.** The Kummer classes are taken at level $N = 2^{61} - 1$, and the program checks that N is prime to $p(q_v - 1)$. That the valuations of the theta values are integers (that $\underline{\underline{q}}$ lies in K_v) holds because $\operatorname{ord} q$ is even at the bad places of these Legendre data; the mockup checks this too and stops if it fails.
- **The two programs and the mockup were all written on the author's side** (with generative AI). The "classical computation" was written from the statement of Theorem 1.10 without IUT, but it is not an implementation by an independent third party. In the C programs and the mockup the extraction of the local data is also shared (in the Python programs the two were extracted independently and agreed).
- **The local data involve choices and assumptions.** $\mathbb{V}_{\mathrm{mod}}^{\mathrm{bad}}$ is the subset of that of the catalogue without the places above 3 and 5. The places above $2, 3, 5, \ell$ are set to 0 in the hull and bounded by Step (ii) and (R4) in the upper bound. The ramification index of K at bad places is assumed by the formula of the catalogue except above p. D2357 has an unfactored cofactor of 1564 digits whose places are not counted.
- **There are seven data; the six completely factored ones have a large margin, and D2357 is conditional.** The family $q \to q^N$ is an adversarial input, but in general it does not consist of initial Θ-data of actual curves. None of the tests tries the claims of IUT.
- **The scope of novelty is narrow.** The view of two computations is in Mochizuki and in the report of LANA, the explicit computation of the IUT side and its constant are in Dupuy–Hilado and Zhou, and the division into a finite layer and a structural layer is in Maeki (§1.3). What is new here is limited to placing the two as programs, the mockup with oracles, and the computer-science reading and position. The search for prior work is as of 5 November 2026 and may have missed things.
- **"The essence" and "not understood essentially" are the position of this note** (H). Mochizuki also speaks of the content of IUT in terms of multiradiality, anabelian geometry, Kummer theory and so on, and our reading of his words is our own. There are opposite positions. Joshi holds that the left end also changes with the reading. Scholze–Stix hold that the proof of Corollary 3.12 has a gap, that if the identifications of the

copies of the real numbers are taken consistently the inequality becomes empty, and that blurring by a factor of at least $O(\ell^2)$ makes it useless [SS, §2.2].

- **The computer-science viewpoint remains a proposal.** The frameworks of equivalence, information flow and relational verification are not formally applied in this note. The check of non-interference is also a test of one hidden choice (the coordinates on Δ_Θ).
- **No line-by-line correspondence can be asked for.** As Mochizuki's caveat (§7) says, explicitly tracing the correspondence between the parts of the two computations is something that cannot be done at all. Our comparison stays at the shape of the whole computation.
- **Sizes such as line counts mean nothing.** The length of a program depends on how it is written and does not measure the difficulty of the computation.
- **lana-agents/iut is moving.** We checked commit `553076a` (3 November 2026, the latest as of 5 November). The plan (`Iut/Multiradial/ROADMAP.md`) is a draft of 14 October 2026.

12 Future work

H The future task is to implement the oracles as actual algorithms, or as approximations to them. In the order of Table 5:

1. `[theta-group]` and `[kummer]`: build the mono-theta environments at a finite level and the Kummer classes of the theta values from finite quotients of tempered fundamental groups, without reverse computation from the ring of the theater. lana-agents/iut has the Heisenberg group, the Kummer classes of the theta function, and parts of the discrete rigidity.
2. `[gab]`: read the type of the abstract group from a finite presentation of the group, not from the field. The recovery of the local invariants is within known results.
3. `[ind3]`: add the iteration of log-links and the log-Kummer correspondence, and make the containment of (Ind3) a computation rather than an assumption.
4. `[global]`: obtain the global data from the global anabelian reconstruction. By the plan of lana-agents/iut, p-adic Hodge theory is needed here.
5. `[step-xi]`: last, Step (xi). After the replacements above, ask whether the output contains the q-pilot of ‡, without reverse computation from the conclusion. As §10 (vi) says, what is needed here is a property of ABC strength.

H As "approximation algorithms" one may think of truncating the infinite constructions to finite ones: a finite level N, finite quotients of profinite groups, finitely many columns of the log-theta-lattice. The questions to ask are then whether the output converges as the truncation is refined, and whether the comparison corresponding to Step (xi) is stable under the truncation. In C or C++, replacing infinite profinite groups by finite objects is a premise, and which finite models are faithful enough is itself a question.

H On the side of tools one can use relational verification, which handles relations over all inputs; type systems of information flow, which enforce multiradiality by types; and formalization in a proof assistant (the plan of lana-agents/iut estimates the formalization of the whole of Theorem 3.11 in Lean at 0.7 to 1.5 million lines). Each of them works as a check when the oracles are removed one by one.

13 Reproducing

The folder `two-computations/` of the bundle contains the C programs (`c/`), the C++ mockup (`cpp/`), the seven local data (`data/`), the scripts (`scripts/`) and the outputs (`outputs/`); its `README.md` lists the commands; `scripts/run_all.sh` rebuilds the programs, rewrites `outputs/` and runs all the checks (about eight minutes), and `paper/make_generated.py` writes the tables and the code excerpts of this note from the outputs and the sources. The C programs compile with `gcc -O2 -Wall -Wextra -std=c11` without warnings; D2357 ($\ell = 157$, long tuples counted by linearity) takes about 10 seconds and the others less than a second. The mockup compiles with `g++ -std=c++17 -O2 -Wall -Wextra -pedantic` without warnings (`make`), and `make check` checks the information flow and the non-interference. `check_c.py` compares the C programs with the Python programs of [RF] (119 items; for D2357 the Python programs add a lower bound 261.75 of the container for the places above the unfactored cofactor, which the C program does not count, so the comparison is made on the counted places), and `check_cpp.py` compares the mockup with the C programs (204 items) and runs the checks of non-interference and the scan of $q \to q^N$. The quotations of Mochizuki's documents, of the report of LANA, of Dupuy–Hilado, Zhou, Maeki and Joshi, and of lana-agents/iut were taken verbatim from the sources and checked by machine, and the mathematical symbols in them on the page images.

Use of generative AI

The question studied in this note — whether placing the computation of IUT and the classical computation side by side as programs lets one approach the content of IUT from the viewpoint of computer science — and the approach of regarding the multiradial algorithm as the essence and placing it in a mockup as oracles arose in the author's discussions with generative AI tools; the author judged them important and decided to make them the subject of this note. Its ultimate aim, to understand exactly the algorithm by which IUT computes the height inequality, is the author's research direction. Generative AI tools (large language models) were used extensively, under the author's direction, in the computations, in writing and checking the programs, and in the writing. The author takes full responsibility for the contents of this note.

A Output of the mockup for D211

```
MOCKUP of the multiradial algorithm of IUT III, Theorem 3.11.
  [implemented]  computed here from the etale-like data, with no access to the rings of the theaters
  [ORACLE name]  not implemented: the value is REVERSE-COMPUTED, from the ring structure of a theater or
                 from the conclusion (ledger at the end). Nothing printed here proves anything.

datum D211  l = 7  k = 4.6666666667
Theta-link dagger -> ddagger: the coric data agree at all 13 places; Theta-pilot(dagger) |-> q-pilot(ddagger); no map
↪  of rings

[implemented] (A)-(B) local invariants from G_{K_v}^ab, and (C) cyclotomic rigidity at level N = 2^61 - 1:
   place          p               d   f     e          rho          dif
   2              2              (above 2, 3, 5 or l: not computed)
   3              3              (above 2, 3, 5 or l: not computed)
   5              5              (above 2, 3, 5 or l: not computed)
   59+            59              1   1     1 0.0000000000 0.0000000000
   59-            59            105   1   105 1.4380952381 0.9904761905  bad: coordinates of Delta_Theta recovered
   ↪  from the commutator
   211+           211           105   1   105 0.9904761905 0.9904761905  bad: coordinates of Delta_Theta recovered
   ↪  from the commutator
   211-           211             1   1     1 0.0000000000 0.0000000000
   80621+         80621           1   1     1 0.0000000000 0.0000000000
   80621-         80621         105   1   105 0.9904761905 0.9904761905  bad: coordinates of Delta_Theta recovered
   ↪  from the commutator
   20139..(21)+   20139..(21)     1   1     1 0.0000000000 0.0000000000
```

```
   20139..(21)-   20139..(21)    105   1   105 0.9904761905 0.9904761905  bad: coordinates of Delta_Theta recovered
   ↪  from the commutator
   14708..(39)+   14708..(39)      1   1     1 0.0000000000 0.0000000000
   14708..(39)-   14708..(39)    105   1   105 0.9904761905 0.9904761905  bad: coordinates of Delta_Theta recovered
   ↪  from the commutator
[implemented] (D) theta values from their Kummer classes at the 5 bad places; j^2-structure: ok
[implemented] (E)-(G) the possible images of the Theta-pilot and the hull, procession-normalized (IUT IV's reading):
   prime 59            places 2  hull 7.4366516239  closed form 7.4366516239
   prime 211           places 2  hull -43.7832965396  closed form -43.7832965396
   prime 80621         places 2  hull 13.0190404504  closed form 13.0190404504
   prime 20139..(21)   places 2  hull 53.8759030973  closed form 53.8759030973
   prime 14708..(39)   places 2  hull 101.2759172208  closed form 101.2759172208
   archimedean container (l+5)/4 log pi = 3.4341896575
[ORACLE ind3]  (Ind3): assumed (the possible images are the containers)

q-pilot      -|log qq|   = -21.8010582065        (log q = 2 l |log qq| = 305.2148148907)
Theta-pilot  -|log Theta| = 135.2584055104       (reading of IUT IV; places above 2, 3, 5, l set to 0)
[ORACLE step-xi]  Step (xi): the q-pilot of ddagger lies in the hull of the possible images: YES  (REVERSE-COMPUTED:
↪  margin 157.0594637169 on this datum)
Corollary 3.12 (from the oracle step-xi): -21.8010582065 <= 135.2584055104
IUT IV       U(D)         = 1145.4012689188       (primes with theta values 295.9301683906, closed form
↪  295.9301683906; 2, 3, 5, l 846.0369108707; archimedean 3.4341896575)
IV: 135.2584055104 <= 1145.4012689188
height inequality (k-1)|log qq| = 79.9372134238 <= 1247.1395405491 = C
Step (viii): 2196.7850225039      Theorem 1.10: (1/6) log q = 50.8691358151 <= 1.548293669782e+08

classical    log q = 305.2148148907  log f = 77.6813935767  log d = 0.8047189562   (all places of bad reduction: log q
↪  = 309.0214773805  log f = 79.5847248216)
classical    Theorem 1.10: (1/6) log q = 50.8691358151 <= 1.548293669782e+08 = right-hand side
the two computations: log q 305.2148148907 vs 2 l |log qq| 305.2148148907; right-hand sides 1.548293669782e+08 vs
↪  1.548293669782e+08

ledger of the oracles (every value REVERSE-COMPUTED):
  gab  G_{K_v}^ab as an abstract group
      reverse-computed from the local fields K_v by local class field theory: (p, [K_v:Q_p], p^f - 1)
  global  global data: weights, prime labels, log d^F, log f^F
      reverse-computed from the number field F and V^bad
  kummer  Kummer classes at level N of the theta values qq^(j^2) (dagger) and of qq (ddagger)
      reverse-computed from the valuations of q in the fields (in IUT: the etale theta function and its evaluation)
  theta-group  the theta groups at level N (the finite-level mono-theta environments)
      built from the theater's own coordinates on Delta_Theta
  ind3  (Ind3), upper semi-compatibility
      assumed, not computed: the images of the Theta-pilot under the log-Kummer correspondence lie in the containers
      ↪  (IUT III, Proposition 3.5; Theorem 3.11 (ii) (a))
  step-xi  Step (xi): the q-pilot of ddagger lies in the hull of the possible images
      reverse-computed from the conclusion: the two log-volumes of this datum were compared. A Step (xi) derived from
      ↪  Theorem 3.11 would give the containment for every initial Theta-datum without comparing numbers
```

```
datum D211: the theaters' coordinates on Delta_Theta (a unit of Z/N per place) are drawn from 8 seeds
  with step (C):    -|log Theta| = 135.258406 135.258406 135.258406 135.258406 135.258406 135.258406 135.258406
  ↪  135.258406
  identical for all 8: yes
  without step (C): -|log Theta| = -5.116062e+18 -2.556386e+18 -4.458824e+18
  depends on the choice: yes; the j^2-structure of the classes is still seen: yes
selftest PASSED
```

```
the family q -> q^N for the datum D211 (N prime to l = 7)
    N       |log qq|     -|log Theta|            U(D)           wall             C   step-xi  height ineq.
    1      21.801058       135.258406      1145.401269      79.937213    1247.139541   yes      holds
    2      43.602116        33.520134      1043.662997     159.874427    1247.139541   yes      holds
    3      65.403175       -75.394850       930.443785     239.811640    1235.658600   no       holds
    4      87.204233      -169.956409       840.186454     319.748854    1247.139541   no       holds
    5     109.005291      -283.310330       726.967242     399.686067    1235.658600   no       holds
    6     130.806349      -380.609665       625.228970     479.623281    1235.658600   no       holds
    8     174.408466      -576.909496       433.233368     639.497707    1247.139541   no       holds
    9     196.209524      -685.824479       320.014155     719.434921    1235.658600   no       holds
   10     218.010582      -792.001688       218.275884     799.372134    1235.658600   no       holds
   11     239.811640      -882.124311       128.018553     879.309348    1247.139541   no       holds
   12     261.612698      -991.039294        14.799340     959.246561    1235.658600   no       holds
   13     283.413757     -1085.600854       -75.457991    1039.183775    1247.139541   no       holds
   15     327.015873     -1322.887730      -290.415474    1199.058201    1235.658600   no       holds
   16     348.816931     -1390.815669      -380.672806    1278.995415    1247.139541   no       fails
   17     370.617990     -1492.553941      -482.411077    1358.932628    1247.139541   no       fails
   18     392.419048     -1601.468924      -595.630289    1438.869842    1235.658600   no       fails
   19     414.220106     -1696.030484      -685.887620    1518.807055    1247.139541   no       fails
   20     436.021164     -1809.384404      -799.106833    1598.744268    1235.658600   no       fails
the oracle step-xi (reverse-computed Step (xi)) says yes up to N = 2, no from N = 3; the height inequality holds up to
↪  N = 15, fails from N = 16
the implemented part (A)-(G) ran the same steps for every N: nothing in it tells these data from genuine ones
```

References

[AbsTopIII] Shinichi Mochizuki, *Topics in absolute anabelian geometry III: global reconstruction algorithms*, J. Math. Sci. Univ. Tokyo **22** (2015), 939–1156.

[Alien] Shinichi Mochizuki, *The mathematics of mutually alien copies: from Gaussian integrals to inter-universal Teichmüller theory*, in: Inter-universal Teichmüller Theory Summit 2016, RIMS Kôkyûroku Bessatsu **B84** (2021), 23–192.

[Cat] Toshihisa Urahigashi, *Explicit initial Θ-data and the two readings of Step (v) of IUT IV, Theorem 1.10: a catalogue*, version 1.0.0, Zenodo, 2026, doi:10.5281/zenodo.23870448.

[DH] Taylor Dupuy and Anton Hilado, *Probabilistic Szpiro, baby Szpiro, and explicit Szpiro from Mochizuki's Corollary 3.12*, arXiv:2004.13108v2 [math.NT], 29 April 2020.

[EssLgc] Shinichi Mochizuki, *On the essential logical structure of inter-universal Teichmüller theory in terms of logical AND "∧"/logical OR "∨" relations: report on the occasion of the publication of the four main papers on inter-universal Teichmüller theory*, RIMS Preprint 1968, November 2022; version of March 2024.

[EtTh] Shinichi Mochizuki, *The étale theta function and its Frobenioid-theoretic manifestations*, Publ. Res. Inst. Math. Sci. **45** (2009), no. 1, 227–349.

[Form] Shinichi Mochizuki, *On the formalization of IUT: a preliminary progress report* (joint work in progress with Y. Hoshi, G. Yamashita, Y. Yang), slides, April 2026, 14 pp., `https://www.kurims.kyoto-u.ac.jp/~motizuki/Formalization%20of%20IUT%20(2026-04%20version).pdf`.

[Gap] Toshihisa Urahigashi, *The gap between the two walls is positive: a closed form for the (Ind2)-terms of lana-agents/iut*, version 1.0.0, Zenodo, 2026, doi:10.5281/zenodo.23954194.

[II] Toshihisa Urahigashi, *Reading the IUT dispute through indexed identifications: an educational model in undergraduate mathematics*, version 2.0.0, Zenodo, 2026, doi:10.5281/zenodo.23933108.

[IUT3] Shinichi Mochizuki, *Inter-universal Teichmüller theory III: canonical splittings of the log-theta-lattice*, Publ. Res. Inst. Math. Sci. **57** (2021), 403–626.

[IUT4] Shinichi Mochizuki, *Inter-universal Teichmüller theory IV: log-volume computations and set-theoretic foundations*, Publ. Res. Inst. Math. Sci. **57** (2021), 627–723.

[Joshi] Kirti Joshi, *Comments on the LANA Project Report of Kato et al.*, 30 July 2026, `https://sites.arizona.edu/kirti-joshi/files/2026/07/Comments-on-the-LANA-Project-Report-of-Kato-et-al.pdf`.

[lana] lana-agents, *iut*: a Lean 4 formalization of inter-universal Teichmüller theory, GitHub repository, `https://github.com/lana-agents/iut`, commit `553076a` (3 November 2026). Files cited: `Iut/WallEquiv/NonDerivable.lean`, `Iut/WallEquiv/README.md`, `Iut/MainTheorem.lean`, `Iut/EtaleTheta/Heisenberg.lean`, `Iut/Multiradial/ROADMAP.md` (a draft of 14 October 2026).

[LANA] Project LANA, *Project LANA interim report on IUT theory*, 20 July 2026, `https://ncatlab.org/nlab/files/LANAProject-Report-July2026.pdf`.

[Maeki] Hidemitsu Maeki, *IUT-Explicit-ADIC*, Zenodo, 22 November 2025, doi:10.5281/zenodo.17677052.

[RF] Toshihisa Urahigashi, *No reading avoids the height inequality: 36 readings of Corollary 3.12 and the conservation of the height load*, version 1.0.0, Zenodo, 2026, doi:10.5281/zenodo.23965952.

[SL] Toshihisa Urahigashi, *Segment or loop? A homotopy-type-theoretic overview of the dispute over Corollary 3.12 of inter-universal Teichmüller theory*, version 1.1.0, Zenodo, 2026, doi:10.5281/zenodo.23933109.

[SS] Peter Scholze and Jakob Stix, *Why abc is still a conjecture*, 2018.

[Stepv] Toshihisa Urahigashi, *A local non-containment in Step (v) of IUT IV, Theorem 1.10*, version 3.0.0, Zenodo, 2026, doi:10.5281/zenodo.23852752.

[WT] Toshihisa Urahigashi, *A programmer's walkthrough of inter-universal Teichmüller theory: running the multiradial algorithm on explicit initial* Θ*-data*, version 1.1.0, Zenodo, 2026, doi:10.5281/zenodo.23955994.

[Zhou] Zhong-Peng Zhou, *The inter-universal Teichmüller theory and new Diophantine results over the rational numbers. I*, arXiv:2503.14510v1 [math.NT], 8 March 2025; code at `https://github.com/zhongpengzhou/IUT-Q-I-code`.